

```

# =====
# WAREX AI vs ML COMPARISON
# Based on the 6 AMRs + 12 Orders from the WAREX project
# Separate analysis for Google Colab
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)

# =====
# 1. WAREX AMRs - SAME 6 AMRs
# =====

amrs = pd.DataFrame({
    "AMR": [
        "AMR-01",
        "AMR-02",
        "AMR-03",
        "AMR-04",
        "AMR-05",
        "AMR-06"
    ],

    # Starting positions from WAREX
    "start_x": [1, 4, 7, 10, 13, 16],
    "start_y": [13, 13, 13, 13, 13, 13],

    # Different battery conditions for comparison
    "battery": [96, 88, 76, 64, 48, 91],

    # Current workload
    "workload": [0, 1, 2, 1, 3, 0]
})

print("WAREX AMR CONFIGURATION")
display(amrs)

# =====
# 2. SAME 12 ORDERS FROM WAREX
# =====

tasks = pd.DataFrame({

    "Order": [
        "ORDER-001",
        "ORDER-002",
        "ORDER-003",
        "ORDER-004",
        "ORDER-005",
        "ORDER-006",
        "ORDER-007",
        "ORDER-008",
        "ORDER-009",
        "ORDER-010",
        "ORDER-011",
        "ORDER-012"
    ],

    "Shelf": [
        "S01", "S05", "S08", "S10",
        "S03", "S12", "S07", "S02",
        "S11", "S06", "S09", "S04"
    ],

    "Destination": [
        "R01", "R02", "D01", "D02",
        "R03", "D03", "R01", "D02",
        "R03", "D01", "R02", "D03"
    ],

    "Priority": [
        3, 2, 3, 2,
        1, 3, 2, 1,
        3, 2, 1, 3
    ]
})

```

```

print("WAREX ORDERS")
display(tasks)

# =====
# 3. WAREHOUSE LOCATIONS
# =====

SHELVES = {
    "S01": (2,2),
    "S02": (5,2),
    "S03": (8,2),

    "S04": (2,5),
    "S05": (5,5),
    "S06": (8,5),

    "S07": (2,8),
    "S08": (5,8),
    "S09": (8,8),

    "S10": (2,11),
    "S11": (5,11),
    "S12": (8,11)
}

PICK_POINTS = {
    "S01": (2,3),
    "S02": (5,3),
    "S03": (8,3),

    "S04": (2,6),
    "S05": (5,6),
    "S06": (8,6),

    "S07": (2,9),
    "S08": (5,9),
    "S09": (8,9),

    "S10": (2,12),
    "S11": (5,12),
    "S12": (8,12)
}

STATIONS = {
    "R01": (13,3),
    "R02": (13,7),
    "R03": (13,11),

    "D01": (16,3),
    "D02": (16,7),
    "D03": (16,11)
}

# =====
# 4. DISTANCE CALCULATION
# =====

def manhattan_distance(p1, p2):
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

# =====
# 5. CREATE AI + ML DATASET
# =====

records = []

for _, amr in amrs.iterrows():
    for _, task in tasks.iterrows():
        start = (
            amr["start_x"],
            amr["start_y"]
        )

        pickup = PICK_POINTS[task["Shelf"]]

        destination = STATIONS[task["Destination"]]

        # Distance from AMR to shelf
        distance_to_shelf = manhattan_distance(

```

```

        start,
        pickup
    )

    # Shelf to delivery/receive station
    distance_to_station = manhattan_distance(
        pickup,
        destination
    )

    total_distance = (
        distance_to_shelf +
        distance_to_station
    )

    records.append({

        "AMR": amr["AMR"],

        "Order": task["Order"],

        "Shelf": task["Shelf"],

        "Destination": task["Destination"],

        "Priority": task["Priority"],

        "Battery": amr["battery"],

        "Workload": amr["workload"],

        "Distance": total_distance,

        "Distance_to_Shelf": distance_to_shelf,

        "Distance_to_Station": distance_to_station
    })

```

```

df = pd.DataFrame(records)

```

```

print("COMBINED WAREHOUSE DATA")
display(df.head(12))

```

```

# =====
# 6. AI RULE-BASED METHOD
# =====

```

```

def AI_score(distance, battery, priority, workload):

```

```

    battery_penalty = max(
        0,
        35 - battery
    ) * 2.5

    workload_penalty = workload * 5

    priority_bonus = priority * 12

    score = (
        distance
        + battery_penalty
        + workload_penalty
        - priority_bonus
    )

    return score

```

```

df["AI_Score"] = df.apply(
    lambda row: AI_score(
        row["Distance"],
        row["Battery"],
        row["Priority"],
        row["Workload"]
    ),
    axis=1
)

```

```

# =====
# 7. CREATE SYNTHETIC TRAINING DATA FOR ML
# =====

```

```

N = 3000

```

```

X_train_data = pd.DataFrame({
    "Distance": np.random.randint(
        5, 80, N
    ),
    "Battery": np.random.randint(
        20, 101, N
    ),
    "Priority": np.random.randint(
        1, 4, N
    ),
    "Workload": np.random.randint(
        0, 7, N
    )
})

# Simulated historical completion time
y_train_data = (
    5
    + X_train_data["Distance"] * 0.85
    + (100 - X_train_data["Battery"]) * 0.10
    + X_train_data["Workload"] * 2.0
    - X_train_data["Priority"] * 2.5
    + np.random.normal(
        0,
        2.5,
        N
    )
)

# =====
# 8. TRAIN ML MODEL
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X_train_data,
    y_train_data,
    test_size=0.20,
    random_state=42
)

ml_model = KNeighborsRegressor(
    n_neighbors=10,
    weights="distance"
)

ml_model.fit(
    X_train,
    y_train
)

# =====
# 9. ML PREDICTION
# =====

features = [
    "Distance",
    "Battery",
    "Priority",
    "Workload"
]

df["ML_Predicted_Time"] = ml_model.predict(
    df[features]
)

# =====
# 10. FIND SELECTED AMR FOR EACH ORDER
# =====

AI_selected = []
ML_selected = []

```

```

for order in tasks["Order"]:

    order_data = df[
        df["Order"] == order
    ]

    # AI selects minimum rule score
    ai_row = order_data.loc[
        order_data["AI_Score"].idxmin()
    ]

    # ML selects minimum predicted completion time
    ml_row = order_data.loc[
        order_data["ML_Predicted_Time"].idxmin()
    ]

    AI_selected.append(
        ai_row["AMR"]
    )

    ML_selected.append(
        ml_row["AMR"]
    )

results = tasks.copy()

results["AI_Selected_AMR"] = AI_selected

results["ML_Selected_AMR"] = ML_selected

print("AI vs ML TASK ALLOCATION")
display(results)

# =====
# GRAPH 1
# AI SCORE FOR ALL 6 AMRs
# =====

avg_ai_score = (
    df.groupby("AMR")["AI_Score"]
    .mean()
    .reindex(amrs["AMR"])
)

plt.figure(figsize=(10,6))

plt.bar(
    avg_ai_score.index,
    avg_ai_score.values
)

plt.title(
    "AI Rule-Based Score – 6 AMRs"
)

plt.xlabel("AMR")

plt.ylabel(
    "Average AI Score"
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

# =====
# GRAPH 2
# ML PREDICTED TIME FOR ALL 6 AMRs
# =====

avg_ml_time = (
    df.groupby("AMR")["ML_Predicted_Time"]
    .mean()
    .reindex(amrs["AMR"])
)

plt.figure(figsize=(10,6))

```

```

plt.bar(
    avg_ml_time.index,
    avg_ml_time.values
)

plt.title(
    "ML Predicted Completion Time - 6 AMRs"
)

plt.xlabel("AMR")

plt.ylabel(
    "Predicted Time (seconds)"
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

# =====
# GRAPH 3
# AI vs ML - 6 AMRs GROUPED BAR
# =====

# Normalize values to 0-100 so the two different units
# can be visually compared without claiming equal units.

ai_norm = (
    (avg_ai_score - avg_ai_score.min())
    /
    (avg_ai_score.max() - avg_ai_score.min() + 1e-9)
)

ml_norm = (
    (avg_ml_time - avg_ml_time.min())
    /
    (avg_ml_time.max() - avg_ml_time.min() + 1e-9)
)

x = np.arange(len(amrs))

width = 0.35

plt.figure(figsize=(12,6))

plt.bar(
    x - width/2,
    ai_norm.values,
    width,
    label="AI Score (normalized)"
)

plt.bar(
    x + width/2,
    ml_norm.values,
    width,
    label="ML Predicted Time (normalized)"
)

plt.xticks(
    x,
    amrs["AMR"]
)

plt.ylabel(
    "Normalized Value"
)

plt.title(
    "AI vs ML - 6 AMR Comparison"
)

plt.legend()

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

```

```

# =====
# GRAPH 4
# BATTERY OF 6 AMRs
# =====

plt.figure(figsize=(10,6))

plt.bar(
    amrs["AMR"],
    amrs["battery"]
)

plt.xlabel("AMR")

plt.ylabel("Battery (%)")

plt.title(
    "Battery Status – 6 AMRs"
)

plt.ylim(
    0,
    100
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

# =====
# GRAPH 5
# WORKLOAD OF 6 AMRs
# =====

plt.figure(figsize=(10,6))

plt.bar(
    amrs["AMR"],
    amrs["workload"]
)

plt.xlabel("AMR")

plt.ylabel("Current Workload")

plt.title(
    "Current Workload – 6 AMRs"
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

# =====
# GRAPH 6
# AI TASK ALLOCATION – PIE CHART
# =====

ai_distribution = (
    results["AI_Selected_AMR"]
    .value_counts()
    .reindex(
        amrs["AMR"],
        fill_value=0
    )
)

plt.figure(figsize=(7,7))

plt.pie(
    ai_distribution.values,
    labels=ai_distribution.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title(
    "AI Task Allocation Across 6 AMRs"
)

```

```

)

plt.show()

# =====
# GRAPH 7
# ML TASK ALLOCATION – PIE CHART
# =====

ml_distribution = (
    results["ML_Selected_AMR"]
    .value_counts()
    .reindex(
        amrs["AMR"],
        fill_value=0
    )
)

plt.figure(figsize=(7,7))

plt.pie(
    ml_distribution.values,
    labels=ml_distribution.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title(
    "ML Task Allocation Across 6 AMRs"
)

plt.show()

# =====
# GRAPH 8
# AI vs ML TASK COUNT – GROUPED BAR
# =====

allocation_compare = pd.DataFrame({

    "AI": ai_distribution,

    "ML": ml_distribution

})

x = np.arange(len(amrs))

plt.figure(figsize=(12,6))

plt.bar(
    x - width/2,
    allocation_compare["AI"],
    width,
    label="AI"
)

plt.bar(
    x + width/2,
    allocation_compare["ML"],
    width,
    label="ML"
)

plt.xticks(
    x,
    amrs["AMR"]
)

plt.xlabel("AMR")

plt.ylabel(
    "Number of Assigned Orders"
)

plt.title(
    "AI vs ML – Task Allocation Across 6 AMRs"
)

plt.legend()

plt.grid(
    axis="y",
    linestyle="dotted",
    color="black",
    alpha=0.3
)

```

```

        alpha=0.3
    )

plt.show()

# =====
# GRAPH 9
# DISTANCE FOR EACH AMR
# =====

avg_distance = (
    df.groupby("AMR")["Distance"]
      .mean()
      .reindex(amrs["AMR"])
)

plt.figure(figsize=(10,6))

plt.bar(
    avg_distance.index,
    avg_distance.values
)

plt.xlabel("AMR")

plt.ylabel(
    "Average Route Distance"
)

plt.title(
    "Average Route Distance – 6 AMRs"
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

# =====
# GRAPH 10
# AI SCORE vs ML PREDICTION SCATTER
# =====

plt.figure(figsize=(9,6))

plt.scatter(
    df["AI_Score"],
    df["ML_Predicted_Time"],
    alpha=0.6
)

plt.xlabel(
    "AI Rule-Based Score"
)

plt.ylabel(
    "ML Predicted Completion Time"
)

plt.title(
    "AI Score vs ML Prediction"
)

plt.grid(
    alpha=0.3
)

plt.show()

# =====
# FINAL SUMMARY
# =====

print("\n" + "="*60)
print("WAREX AI vs ML SUMMARY")
print("="*60)

print("\n6 AMRs:")
print(", ".join(amrs["AMR"]))

print("\nTotal Orders:", len(tasks))

```

```
print(
    "\nAI task allocation:"
)

print(ai_distribution)

print(
    "\nML task allocation:"
)

print(ml_distribution)

print("\nML MODEL PERFORMANCE")

ml_test_predictions = ml_model.predict(X_test)

mae = mean_absolute_error(
    y_test,
    ml_test_predictions
)

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        ml_test_predictions
    )
)

r2 = r2_score(
    y_test,
    ml_test_predictions
)

print(
    f"MAE : {mae:.2f} seconds"
)

print(
    f"RMSE : {rmse:.2f} seconds"
)

print(
    f"R² : {r2:.3f}"
)

print("\nAI uses explicit warehouse rules.")
print("ML predicts completion time from learned examples.")
```


WAREX AMR CONFIGURATION

	AMR	start_x	start_y	battery	workload	
0	AMR-01	1	13	96	0	
1	AMR-02	4	13	88	1	
2	AMR-03	7	13	76	2	
3	AMR-04	10	13	64	1	
4	AMR-05	13	13	48	3	
5	AMR-06	16	13	91	0	

WAREX ORDERS

	Order	Shelf	Destination	Priority	
0	ORDER-001	S01	R01	3	
1	ORDER-002	S05	R02	2	
2	ORDER-003	S08	D01	3	
3	ORDER-004	S10	D02	2	
4	ORDER-005	S03	R03	1	
5	ORDER-006	S12	D03	3	
6	ORDER-007	S07	R01	2	
7	ORDER-008	S02	D02	1	
8	ORDER-009	S11	R03	3	
9	ORDER-010	S06	D01	2	
10	ORDER-011	S09	R02	1	
11	ORDER-012	S04	D03	3	

COMBINED WAREHOUSE DATA

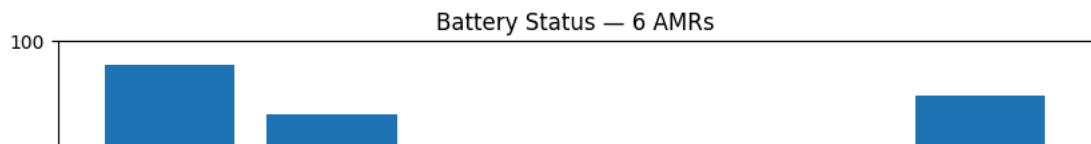
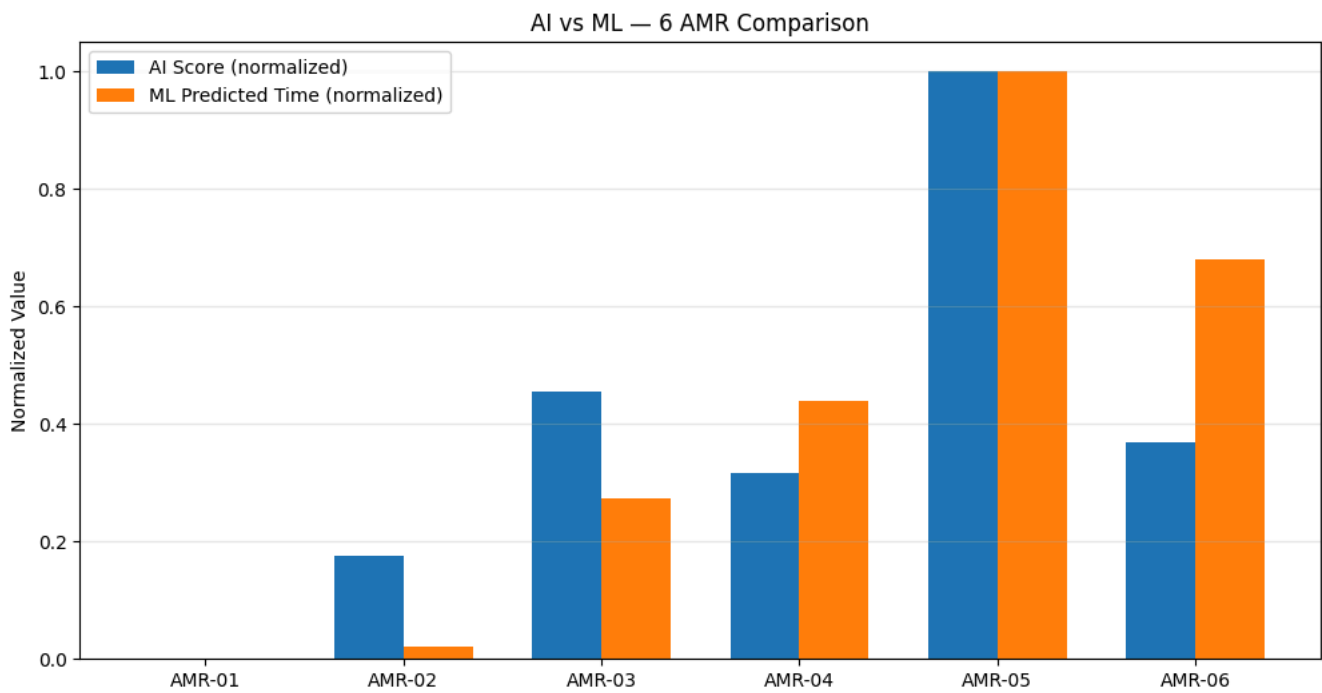
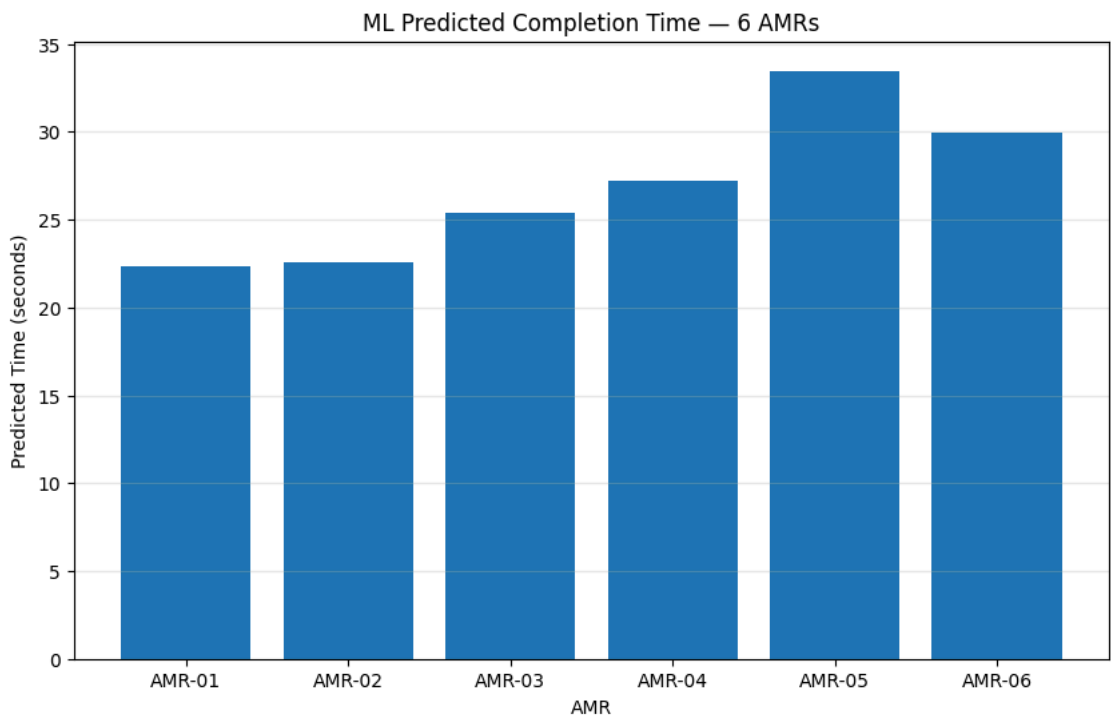
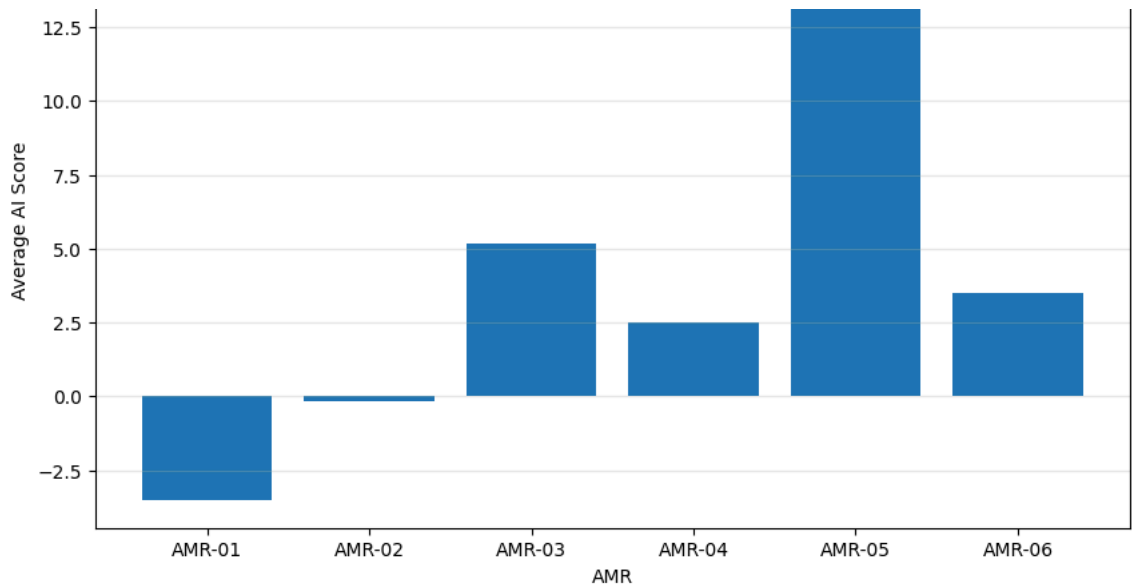
	AMR	Order	Shelf	Destination	Priority	Battery	Workload	Distance	Distance_to_Shelf	Distance_to_Station
0	AMR-01	ORDER-001	S01	R01	3	96	0	22	11	11
1	AMR-01	ORDER-002	S05	R02	2	96	0	20	11	9
2	AMR-01	ORDER-003	S08	D01	3	96	0	25	8	17
3	AMR-01	ORDER-004	S10	D02	2	96	0	21	2	19
4	AMR-01	ORDER-005	S03	R03	1	96	0	30	17	13
5	AMR-01	ORDER-006	S12	D03	3	96	0	17	8	9
6	AMR-01	ORDER-007	S07	R01	2	96	0	22	5	17
7	AMR-01	ORDER-008	S02	D02	1	96	0	29	14	15
8	AMR-01	ORDER-009	S11	R03	3	96	0	14	5	9
9	AMR-01	ORDER-010	S06	D01	2	96	0	25	14	11
10	AMR-01	ORDER-011	S09	R02	1	96	0	18	11	7
11	AMR-01	ORDER-012	S04	D03	3	96	0	27	8	19

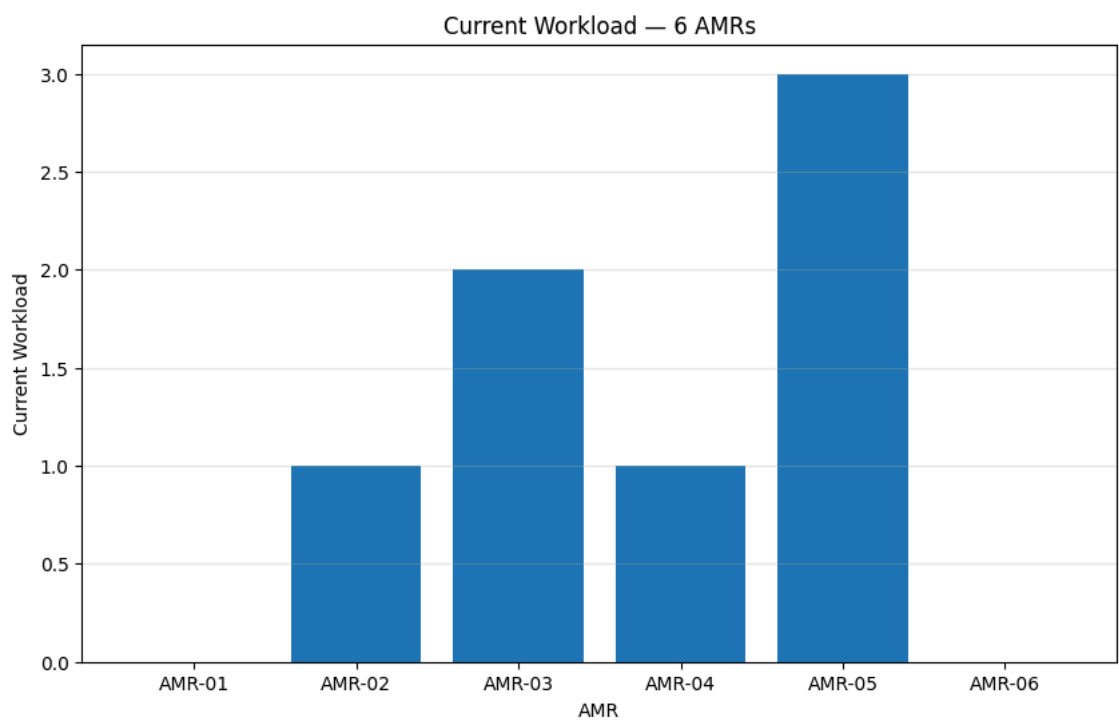
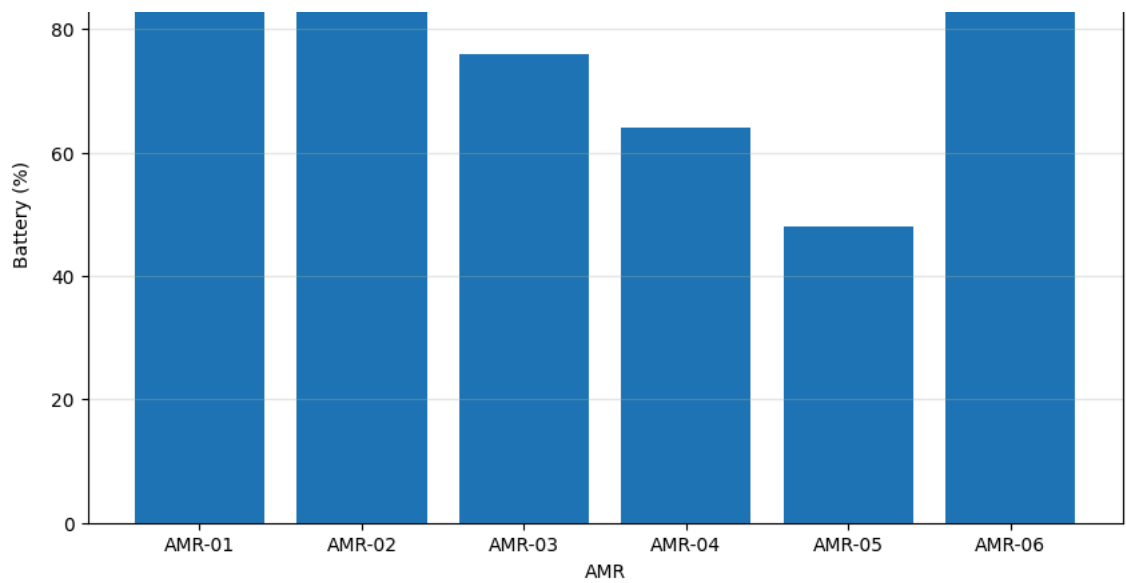
AI vs ML TASK ALLOCATION

	Order	Shelf	Destination	Priority	AI_Selected_AMR	ML_Selected_AMR	
0	ORDER-001	S01	R01	3	AMR-01	AMR-01	
1	ORDER-002	S05	R02	2	AMR-01	AMR-01	
2	ORDER-003	S08	D01	3	AMR-01	AMR-02	
3	ORDER-004	S10	D02	2	AMR-01	AMR-01	
4	ORDER-005	S03	R03	1	AMR-01	AMR-02	
5	ORDER-006	S12	D03	3	AMR-01	AMR-01	
6	ORDER-007	S07	R01	2	AMR-01	AMR-01	
7	ORDER-008	S02	D02	1	AMR-01	AMR-02	
8	ORDER-009	S11	R03	3	AMR-01	AMR-02	
9	ORDER-010	S06	D01	2	AMR-01	AMR-03	
10	ORDER-011	S09	R02	1	AMR-01	AMR-04	
11	ORDER-012	S04	D03	3	AMR-01	AMR-01	

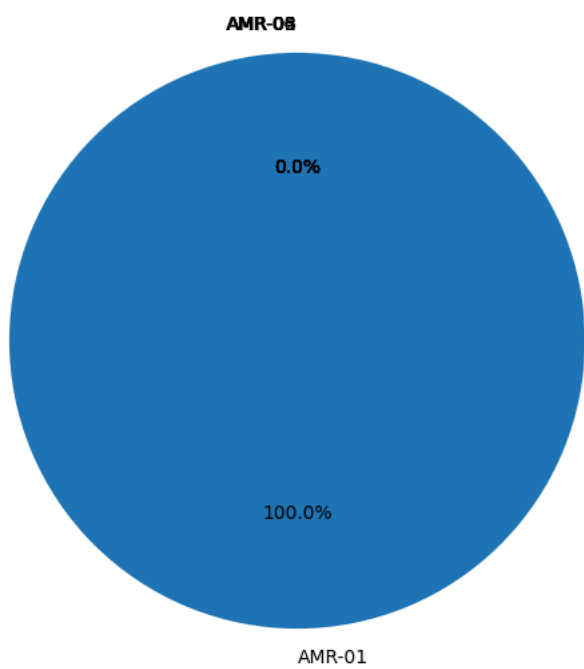
AI Rule-Based Score — 6 AMRs





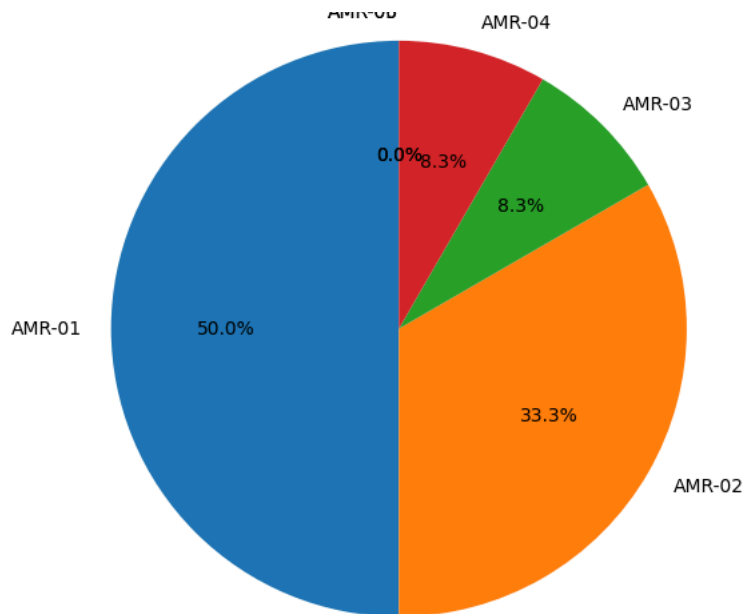


AI Task Allocation Across 6 AMRs



ML Task Allocation Across 6 AMRs

AMR-05



AI vs ML — Task Allocation Across 6 AMRs

